

Property B: The LLM Writes Under Orchestration Rules — A Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 2 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the five mediator properties named in the source paper's AI-as-substrate-mediator commitment — **Property B: the LLM writes under orchestration rules** — as a standalone architectural commitment with independent operational content, separable from the four other mediator properties (read-as-primary-state, no-state-outside-substrate, no-authority, write-attribution) with which it composes into the integrated mediator role.

Abstract

The CKS pattern's AI-as-substrate-mediator commitment names five properties as jointly defining the architectural role LLMs play in CKS systems. A separate note formalizes that joint commitment as a single role; a parent-frame note establishes the five properties as severable architectural commitments. This note formalizes one of those five — Property B, the commitment that LLM-produced substrate writes happen under orchestration rules humans authored at design time — as having independent operational content that can be defended, implemented, and tested independently of the others. Implementations under pressure to support agentic LLM behavior or high-throughput LLM operations consistently drift toward writes that escape rule-governance; naming Property B as standalone gives downstream implementers a precise specification of the architectural commitment to rule-governed LLM writes, independent of how the other mediator properties are realized. The note states the five operational components of Property B, distinguishes it from four adjacent patterns commonly conflated with it (autonomous agent writes, prompt-pattern-emergent writes, configuration-driven writes, ML-policy-trained writes), names eight failure modes that violate it, traces its load-bearing connections to other CKS commitments, and provides an operational test for whether a system's LLM write behavior is CKS-coherent specifically on this axis.

1. Why Property B needs to be formalized as standalone

The parent foundational note on AI-as-substrate-mediator commits to the integrated mediator role and names five properties — read-as-primary-state, write-under-rules, no-state-outside-substrate,

no-authority, write-attribution — as jointly characterizing it. A parent-frame decomposition note establishes that the five are severable architectural commitments: each has independent operational content, each can fail or be satisfied separately, and each merits standalone formalization. This note carries that work forward for the second of the five — Property B, the commitment that when an LLM operating within a CKS cell produces outputs that affect substrate state, those writes happen under orchestration rules humans authored at design time.

The motivating cases are implementations that produce LLM substrate writes through patterns other than rule-governed writes: autonomous-agent deployments where the agent decides what to write based on emergent goal-pursuit; prompt-pattern deployments where the LLM writes whenever the prompt's pattern triggers; configuration-driven deployments holding write rules in deployment files outside substrate; and ML-policy deployments where a trained policy determines which writes happen. In each, the LLM's writes are authorized by something other than a substrate-resident human-authored rule — and therefore by something outside the human-governed substrate the architecture commits to as the seat of authority.

A second motivation is the strategic prior-art posture: patentable derivations of CKS focused on LLM write architectures are substantially more defensibly contested when Property B is publicly formalized as standalone. A third is the labor allocation framework the source paper introduces in §2.3, which names three modes of work — direct human labor, LLM labor under orchestration rules, and stable-cell automation. Property B is what makes the second mode architecturally specific; without it, LLM-under-rule operation is indistinguishable at the architecture's authority layer from autonomous-agent operation, and the framework's typology collapses.

2. The Property B commitment, defined precisely

In the CKS pattern, a system satisfies **Property B** if and only if all of the following hold during LLM execution within cells.

(a) Rule-authorized writes only. Every substrate write produced by LLM operations happens under the authorization of an orchestration rule. The rule specifies, at minimum, that writes of this kind are permitted, when they may happen, and what content requirements they must satisfy. LLM writes outside rule authorization are not architectural writes in the CKS sense.

(b) Rules-as-substrate-content. The orchestration rule that authorizes the write is itself substrate content. The rule lives in the substrate, addressable with its own provenance, governable through the three rights humans hold over substrate content. Rules held outside substrate — in code, configuration files, prompt templates external to substrate, or vendor-managed runtime settings — do not satisfy this component, because the rule itself is then not subject to the governance architecture the substrate provides. This component is what distinguishes architectural rule-governance from procedural rule-application.

(c) Authorization at write time, not post-hoc. The rule's authorization is exercised at the moment the write is committed; it is not authorized retroactively by rules created after the write or by audit logging that characterizes the write as if it had been rule-governed. A system that commits writes first and

attaches rule references later does not satisfy Property B.

(d) Human-authored rules. The rule was authored at design time by humans — specifically, the rule's commit to the substrate was a human exercise of authority. Rules generated by the LLM at execution time, by another LLM, or by automated processes without a human authority in the commit do not satisfy this component. LLM-drafted rules subject to human authority before they take effect remain admissible; LLM-committed rules outside human authority are not.

(e) Content satisfies rule requirements. The LLM's write content satisfies the rule's content requirements. If the rule specifies that writes must contain certain fields, follow certain formats, or maintain certain invariants, the LLM's write satisfies those requirements. The rule does not just authorize the act of writing; it specifies what writes are.

The five together define what Property B requires architecturally. A system that satisfies fewer than all five produces LLM writes that depart from rule-governance in some specific way and breaks the architectural commitment in that direction.

3. What Property B does NOT require

Stating precisely what Property B does not require keeps the framing inside what the source paper supports.

It does not require rules to be exhaustive. Rules cover what they cover; situations not addressed by any rule are handled by direct override or by humans operating outside cells. Property B requires that every LLM-produced substrate write happens under whichever rule authorizes it, not that every conceivable situation has a corresponding rule.

It does not require rules to be deterministic. Rules may specify probabilistic behaviors, LLM-judgment-within-constraints, or non-deterministic processes. The architectural commitment is to rule-governance, not to deterministic outcomes.

It does not require rules to be expressed in any specific form. Rules may be written in natural language, structured DSL, code, or prompt templates that themselves live in substrate. What the architecture requires is that rules are substrate content humans authored, not that rules conform to a specific representation.

It does not require rules to be applied uniformly across cells. Different cells may operate under different rules, and the same cell may operate under different rules at different times — rules are themselves modifiable substrate content. The architectural commitment is that whatever rule is in effect for a given cell execution is the rule that authorizes the LLM's writes during that execution.

It does not require single-rule authorization. Multiple rules may interact to authorize a write — one specifying the trigger, another the content requirements, another the conflict-resolution policy. What the architecture requires is that the conjunction of applicable rules authorizes the write.

4. What Property B is NOT

Four adjacent patterns are commonly conflated with Property B. Each is a real and reasonable pattern in some other architecture.

Not autonomous agent writes. Autonomous agents in agent frameworks decide what to write based on goal-pursuit, reasoning, and emergent behavior over the course of a multi-step session. Property B is different: LLM writes happen under rule authorization, not under agent judgment. A CKS deployment that uses an agent framework as the cell's execution mechanism must constrain the agent to rule-governed writes only; the framework's native autonomous-write paths violate Property B because they lack the rule-authorization component. This is the most consequential conflation in current AI infrastructure, because agent-framework idioms are well-tooled, familiar to practitioners, and silently substitute agent judgment for orchestration-rule authorization at the write layer.

Not prompt-pattern-emergent writes. Some implementations produce LLM writes through prompt patterns: the prompt is designed so that the LLM, when processing certain inputs, produces certain outputs that are then committed as substrate writes. The prompt design is not the rule; the rule is the substrate-resident authorization that specifies the write is permitted. A system whose only "rule" is the prompt pattern fails Property B because the prompt is not substrate content. Prompt patterns may be the technical mechanism by which substrate-resident rules are realized at execution time; what cannot happen is the prompt pattern substituting for the rule.

Not configuration-driven writes. Configuration-driven systems specify write rules in deployment configurations: which cells can write what to which substrate locations, under which conditions. The configurations may be human-authored, but Property B requires the rules to be substrate content, not deployment artifacts. Configurations are typically not subject to the three rights humans hold over substrate content — modifying a configuration is a different governance action than modifying a substrate-resident rule, and conflating the two breaks the human-governed commitment at the write layer.

Not ML-policy-trained writes. Some implementations train ML policies (separate from the LLM mediator) that determine which writes happen based on learned patterns from training data. The trained policy is not an orchestration rule; it is a parametric representation of learned behavior, not human-authored substrate content. ML-trained components can be used as adjacent components within cells under hybrid composition patterns, but the cell's write authorization must come from substrate-resident orchestration rules, not from the trained policy.

5. Why Property B is load-bearing for downstream commitments

The connections below are not new commitments; they name the work Property B does in making other CKS commitments operationally coherent.

Human-governed. Human governance at the write layer is operationalized through orchestration rules humans authored. Property B is what makes that operationalization architecturally specific — every LLM write traces to a rule humans authored, with the rule itself subject to the three rights. Without Property B, LLM writes happen on the LLM's authority, breaking the human-governed commitment at the most consequential moment for substrate state.

Cell-level conflict resolution. Cells resolve contradictions under orchestration rules; this is one specific application of Property B at the conflict-handling layer. The two-level conflict structure the source paper commits to in §5.2 — substrate-level preservation plus cell-level resolution under rules — depends on Property B holding the cell-level layer, so that resolution decisions are rule-authorized substrate writes rather than LLM-judgment-driven writes that happen to overlap with what a rule would have specified.

Labor allocation framework. The LLM-under-rule mode the source paper names in §2.3 is defined by Property B. Without it, the mode collapses into direct human labor wearing an LLM costume, or into autonomous-agent operation outside the framework's three modes entirely.

Path retraceability. LLM-mediated writes are retraceable because their provenance includes the rule reference. Property B is what makes the rule reference architecturally specific — there is always a rule that authorized the write, and the rule itself is substrate content with its own provenance. A system without Property B has writes whose rule references are absent, post-hoc, or fabricated; retraceability becomes a procedural commitment rather than an architectural one.

Architectural-property qualifier on governance. Governance in CKS is a property of architecture, not of process. Property B contributes by making LLM-write-governance architectural: the rule is substrate content, the rule's authorization is at write time, the rule's authorship is human at design time. None depends on procedural compliance; all are architectural.

6. Failure modes that violate Property B

A system can fail Property B specifically while satisfying other mediator properties. Eight failure modes name the most common ways this happens.

(a) Autonomous LLM writes. The LLM produces substrate writes based on its own judgment, without an orchestration rule authorizing the write. The write may be helpful or accurate; it is not architecturally rule-governed.

(b) Prompt-pattern-only writes. The deployment uses prompt patterns to elicit LLM writes, with no substrate-resident rule authorizing them. The prompt design is the only "authorization"; the rules-as-substrate-content component is violated.

(c) Configuration-driven writes outside substrate. Write rules are held in configuration files, environment variables, or runtime settings outside substrate. The rules may be human-authored, but they are not substrate content and therefore not subject to the three rights.

(d) Post-hoc rule attribution. The LLM produces a write, and the system later identifies which rule "would have authorized" it, attaching the rule reference as post-hoc provenance. The rule's authorization was not in effect at write time; the at-write-time-authorization component is violated.

(e) LLM-generated rules. The LLM generates orchestration rules that authorize its own subsequent writes. The rules may be substrate content with provenance; what makes them non-CKS-coherent is that they were not human-authored in the architectural sense.

(f) ML-policy authorization. A trained ML policy (separate from the LLM) determines which writes happen, with the LLM executing the writes under the policy's direction. The policy is not an orchestration rule; the LLM's writes are policy-authorized rather than rule-authorized, even when the policy was trained under careful human supervision.

(g) Writes that bypass rule content requirements. The orchestration rule authorizes the write and specifies content requirements (specific fields, formats, invariants), but the LLM's write fails the requirements while still being committed. The rule's authorization was in effect, but the rule's content requirements were not satisfied.

(h) Rule-bypassing fast paths. The deployment includes optimization paths where LLM writes happen without consulting the rule layer — for performance reasons, to handle high-throughput scenarios, or to support real-time interaction patterns. Property B is an architectural commitment, not a default-path commitment; a fast path that bypasses it violates the architecture.

7. Operational test

A system satisfies Property B if and only if all of the following are true at all times during LLM execution within cells:

1. Every substrate write produced by LLM operations is authorized by an orchestration rule that is in effect at the moment the write is committed.
2. The orchestration rule that authorizes the write is substrate content — addressable in the substrate, governable through the three rights, with its own provenance.
3. The orchestration rule was authored at design time by humans, with the rule's commit being a human exercise of authority rather than an LLM- or policy-generated artifact.
4. The LLM's write content satisfies the rule's content requirements (fields, formats, invariants) as the rule specifies them.
5. No LLM-produced substrate write happens outside rule authorization — including writes through fast paths, optimization shortcuts, or autonomous-agent paths that bypass the rule layer.

A system that fails any of (1)–(5) does not satisfy Property B in the architectural sense, even when its other mediator properties are preserved.

8. Why naming Property B as standalone matters

Implementations under pressure to support agentic LLM behavior or high-throughput LLM operations consistently drift toward writes that happen outside rule-governance. The drift is steady because rule-governance feels architecturally restrictive compared to autonomous patterns, and because agent-framework idioms — well-tooled, familiar, and natively oriented toward autonomous writes — are the path of least resistance. Implementations that drift produce systems where LLM writes appear to happen "in the system" but escape the architecture's authorization structure, manifesting downstream as governance failures, retraceability gaps, and labor-allocation framework collapse. Each consequence is recoverable in principle by procedural patches; none is recoverable architecturally without restoring

rule-governance at the write layer itself.

Property B pairs with the no-authority property — the commitment that the LLM does not exercise authority over substrate content — on the write axis, and the two are independently severable. A system can fail Property B while satisfying no-authority (LLM writes happen autonomously but never claim authority — which still violates the architectural commitment to rule-governed writes), and a system can fail no-authority while satisfying Property B (LLM writes happen under rules but the rules grant authority that should remain human — which violates a different architectural commitment). Property B is the positive specification of how LLM writes are to happen — under rule authorization. The no-authority property is the negative specification of what LLM writes cannot do — exercise authority humans hold. The standalone treatment of each is what allows them to be defended, implemented, and tested independently; the joint treatment in the parent foundational note is what assembles them, alongside the other three, into the integrated mediator role.

Subsequent work that implements, extends, or argues against the CKS LLM-write commitment should use "Property B" in the sense formalized here. Subsequent work that uses the term differently — or that treats LLM writes as authorized by mechanisms other than substrate-resident human-authored rules — is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Property B: The LLM Writes Under Orchestration Rules — A Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern*. May 2 2026. ORCID: 0009-0004-8065-3235.